1.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
df = pd.read_csv(r'housing.csv)
df.head()
df.info()
df.nunique()
df.isnull().sum()

df.duplicated().sum()
df['total_bedrooms'].median()
df['total_bedrooms'].fillna(df['total_bedrooms'].median(),
inplace=True)

df.describe()
df.describe().T
Numerical = df.select_dtypes(include=[np.number]).columns
print(Numerical)

for col in Numerical:
    plt.figure(figsize=(10, 6))
    df[col].plot(kind='hist', title=col, bins=60, edgecolor='black')
    plt.ylabel('Frequency')
    plt.show()

for col in Numerical:
    plt.figure(figsize=(6, 6))
    sns.boxplot(df[col], color='blue')
    plt.title(col)
    plt.ylabel(col)
    plt.show()
```

2.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(r'housing.csv')

df.head()
df.isnull().sum()
x=df['total_bedrooms'].median()

df['total_bedrooms'].fillna(x,inplace=True)
df1=df.drop(['longitude','latitude','ocean_proximity'],axis=1)
df1.head()

plt.figure(figsize=(10, 6))
corr_matrix = df1.corr()
print(corr_matrix)

sns.heatmap(corr_matrix,cmap='Reds',annot=True)
plt.title("Feature Correlation Heatmap")
plt.show()

sns.pairplot(df1)
plt.show()
```

```
3.from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd
import matplotlib.pyplot as plt

iris = load_iris()
x= iris.data
y= iris.target
print(x[1:5,:])
print(y[1:10])

iris_df=pd.DataFrame(data=features,columns=['Sepal Length', 'Sepal
Width', 'Petal Length', 'Petal Width'])
iris_df['Target']=target
iris_df.head()

x_scaled =StandardScaler().fit_transform(x)
print(x_scaled[1:5,:])

pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
x_scaled=Standard scaler().fit_transform(x)
Print(x_pca[1:5,:])

explained_variance=x_pca
print(“explained variance by PC1 : {explained_variance[0]:.2f}”)
print(“explained variance by PC2 : {explained_variance[1]:.2f}”)

colors = ['red', 'green', 'blue']
labels = iris.target_names
plt.figure(figsize=(8, 6))
for i in range(len(colors)):
   plt.scatter(x_pca[y== i, 0], x_pca[y== i, 1], color=colors[i],
label=labels[i])
```

```
plt.xlabel(“PC1”)
plt.ylabel(“PC2”)
plt.title(‘PCA on iris dataset (dimensionality reduction)’)
plt.legend()
plt.grid()
plt.show()

4.import pandas as pd

data = pd.read_csv(r"training_data.csv")
print(data)

def find_s_algorithm(data):
    """Implements the Find-S algorithm to find the most specific
hypothesis."""

    attributes = data.iloc[:, :-1].values
    target = data.iloc[:, -1].values

    for i in range(len(target)):
        if target[i] == "Yes":
            hypothesis = attributes[i].copy()
            break

    for i in range(len(target)):
        if target[i] == "Yes":
            for j in range(len(hypothesis)):
                if hypothesis[j] != attributes[i][j]:
                    hypothesis[j] = '?'  # Generalize inconsistent attributes
        print(i,hypothesis)
    return hypothesis
# Run/call Find-S Algorithm
final_hypothesis = find_s_algorithm(data)
# Print the learned hypothesis
print("Most Specific Hypothesis:", final_hypothesis)
```

```
5.import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

x_values = np.random.rand(100)
labels = np.array(["Class1" if x <= 0.5 else "Class2" for x in
x_values[:50]])

X_train=x_values[:50].reshape(-1,1)
y_train=labels

df1=pd.DataFrame({"Points":[f"x{i+1}" for i in
range(50)],"Value":x_values[:50],"Label":labels})
print(df1)
X_test=x_values[50:].reshape(-1,1)
true_labels = np.array(["Class1" if x <= 0.5 else "Class2" for x in
x_values[50:]])

k_values = [1, 2, 3, 4, 5, 20, 30]
results = {}
accuracies = {}
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    predictions = knn.predict(X_test)
    results[k] = predictions

    accuracy = accuracy_score(true_labels, predictions) * 100
    accuracies[k] = accuracy
    print(f"Accuracy for k={k}: {accuracy:.2f}%")

df2=pd.DataFrame({"Points":[f"x{i+1}" for i in range(50,100)],"actual
labels":true_labels,"Predicted Labels": results[2]})
print(df2)
```

```
6.import numpy as np
import matplotlib.pyplot as plt

def gaussian_kernel(x, x_query, tau):
    return np.exp(- (x - x_query) ** 2 / (2 * tau ** 2))

def locally_weighted_regression(X, y, x_query, tau):
    X_b = np.c_[np.ones(len(X)), X]  # Add bias term (Intercept)
    x_query_b = np.array([1, x_query])  # Query point with bias term
    W = np.diag(gaussian_kernel(X, x_query, tau))  # Compute weights
    # Compute theta: (X^T W X)^-1 X^T W y
    theta = np.linalg.inv(X_b.T @ W @ X_b) @ X_b.T @ W @ y
    return x_query_b @ theta  # Return prediction

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y = np.array([1, 3, 2, 4, 3.5, 5, 6, 7, 6.5, 8])

X_query = np.linspace(1, 10, 100)

tau = 1.0
y_lwr = np.array([locally_weighted_regression(X, y, x_q, tau) for x_q
in X_query])

plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Data Points')
plt.plot(X_query, y_lwr, color='red', label='Locally Weighted
Regression')
plt.title("Locally Weighted Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

7A

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

data=pd.read_csv(r"Boston housing dataset - Copy.csv")
data.head()
data.shape
data.info()
data.isnull().sum()
df=data.copy()

df['CRIM'].fillna(df['CRIM'].mean(),inplace=True)
df['ZN'].fillna(df['ZN'].mean(),inplace=True)
df['CHAS'].fillna(df['CHAS'].mode()[0],inplace=True)
df['INDUS'].fillna(df['INDUS'].mean(),inplace=True)
df['AGE'].fillna(df['AGE'].median(),inplace=True)
df['LSTAT'].fillna(df['LSTAT'].median(),inplace=True)
df.isnull().sum()
df.head()

X=df.drop('MEDV',axis=1)
y=df['MEDV']

scale=StandardScaler()
X_scaled=scale.fit_transform(X)
X_train,X_test,y_train,y_test=train_test_split(X_scaled,y,test_size=0.2,
random_state=42)
```

```
model=LinearRegression()

model.fit(X_train,y_train)

y_pred=model.predict(X_test)
y_pred

mse=mean_squared_error(y_test,y_pred)
rmse=np.sqrt(mse)

r2=r2_score(y_test,y_pred)
print(f'Mean Squared Error:{mse}')
print(f'Root Mean Squared Error:{rmse}')
print(f'R-squared:{r2}')

#7b
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

data=pd.read_csv(r'auto-mpg.csv')
data.head()
data.shape
data.info()
data.isnull().sum()
df=data.copy()

df['horsepower'].fillna(df['horsepower'].median(),inplace=True)
```

```
X=df[['horsepower']]
y=df['mpg']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random
_state=42)

degree=2
poly=PolynomialFeatures(degree)
X_poly_train=poly.fit_transform(X_train)

model=LinearRegression()
model.fit(X_poly_train,y_train)

X_poly_test=poly.transform(X_test)
y_pred=model.predict(X_poly_test)

plt.scatter(X,y,color='blue',label='Data')
X_range=np.linspace(X.min(),X.max(),100)
X_range_poly=poly.transform(X_range)
y_range_pred=model.predict(X_range_poly)
plt.plot(X_range,y_range_pred,color='red',label='Polynomial Fit')
plt.xlabel('Horsepower')
plt.ylabel('MPG')
plt.legend()
plt.title(f'Polynomial Regression(degree{degree})')
plt.show()

mse=mean_squared_error(y_test,y_pred)
rmse=np.sqrt(mse)
r2=r2_score(y_test,y_pred)

print(f'Mean Squared Error(MSE):{mse:.2f}')
print(f'Root Mean Squared Error(RMSE):{rmse:.2f}')
print(f'R-squared(R**2):{r2:.2f}')
```

```
8
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

from sklearn.tree import export_graphviz
from IPython.display import Image
import pydotplus

import warnings
warnings.filterwarnings('ignore')

data = pd.read_csv(r'Breast Cancer Dataset.csv')
data.head()
data.shape
data.info()
data.diagnosis.unique()
data.isnull().sum()
df = data.drop(['id'], axis=1)
df['diagnosis'] = df['diagnosis'].map({'M':1, 'B':0}) # Malignant:1,
Benign:0
X = df.drop('diagnosis', axis=1)  # Drop the 'diagnosis' column (target)
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = DecisionTreeClassifier(criterion='entropy') #criteria = gini,
entropy
```

```
model.fit(X_train, y_train)
model
y_pred = model.predict(X_test)
y_pred

accuracy = accuracy_score(y_test, y_pred) * 100
classification_rep = classification_report(y_test, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:\n", classification_rep)
new = [[12.5, 19.2, 80.0, 500.0, 0.085, 0.1, 0.05, 0.02, 0.17, 0.06,
    0.4, 1.0, 2.5, 40.0, 0.006, 0.02, 0.03, 0.01, 0.02, 0.003,
    16.0, 25.0, 105.0, 900.0, 0.13, 0.25, 0.28, 0.12, 0.29, 0.08]]
y_pred = model.predict(new)

if y_pred[0] == 0:
   print("Prediction: Benign")
else:
   print("Prediction: Malignant")
dot_data=export_graphviz(model,feature_names=X_train.columns,fill
ed=True,rounded=True)

graph = pydotplus.graph_from_dot_data(dot_data)

Image(graph.create_png())

plt.figure(figsize=(12, 8))
plot_tree(model, filled=True, feature_names=X.columns,
class_names=['Benign', 'Malignant'], rounded=True)
plt.show()
```

9

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = np.load('olivetti_faces_offline.npz')
images = data['images']
targets = data['target']

plt.figure(figsize=(10, 8))
for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(images[i], cmap='gray')
    plt.title(f"ID: {targets[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

#X = images.reshape((400, 4096))
X = images.reshape((images.shape[0], -1))  # (400, 4096)
y = targets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
stratify=y, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy : {accuracy * 100:.2f}%")
```